

UNIVERSIDADE FEDERAL DE RORAIMA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
ANÁLISE DE ALGORITMOS

**Algoritmo Genético Híbrido com Operador de Reparo Heurístico Aplicado ao
Problema da Cobertura Mínima de Vértices**

Discentes: Arthur Correia De Oliveira Ramos , Yan Dos Santos Teixeira

Professor(a): Professor Doutor Hebert Oliveira Rocha

Boa Vista
2026

Resumo

O Problema da Cobertura Mínima de Vértices (*Minimum Vertex Cover* – MVC) consiste em determinar, para um grafo $G = (V, E)$, o menor subconjunto de vértices $S \subseteq V$ tal que toda aresta de E possua ao menos uma extremidade em S . Trata-se de um problema NP-difícil, o que motiva o uso de metaheurísticas para sua resolução em instâncias de porte considerável. Este trabalho apresenta a implementação e a análise de um Algoritmo Genético (AG) híbrido, dotado de um operador heurístico de reparo responsável por garantir a viabilidade de todos os indivíduos da população ao longo da evolução. O algoritmo foi avaliado sobre um grafo sintético gerado pelo modelo de Barabási-Albert, contendo 50 vértices e 141 arestas, concebido como uma representação simplificada de uma malha urbana de monitoramento, na qual os vértices correspondem a interseções viárias e as arestas a trechos de via a serem cobertos por ativos de segurança. Após 150 gerações, com população de 50 indivíduos, o algoritmo convergiu para uma cobertura de tamanho 28, correspondente a 56% do total de vértices do grafo. Os resultados evidenciam o comportamento típico de algoritmos genéticos aplicados a problemas combinatórios: melhoria rápida nas gerações iniciais, seguida de um longo platô de estagnação e uma melhoria tardia viabilizada pela manutenção de diversidade populacional via mutação. Conclui-se que a hibridização entre busca evolutiva e heurística gulosa de reparo constitui uma estratégia eficaz para garantir soluções factíveis de boa qualidade, sem, contudo, eliminar o risco de estagnação em ótimos locais.

Palavras-chave: Algoritmos Genéticos. Cobertura Mínima de Vértices. Otimização Combinatória. Redes Complexas. Metaheurísticas.

Sumário

Resumo	1
1 Introdução	3
2 Fundamentação Teórica	3
2.1 O Problema da Cobertura Mínima de Vértices	3
2.2 Algoritmos Genéticos	4
2.3 Redes Livres de Escala e o Modelo de Barabási-Albert	4
3 Metodologia	4
3.1 Codificação e Função de Aptidão	5
3.2 Parâmetros Experimentais	5
3.3 Instância de Teste	5
4 Implementação	6
4.1 Operador de Reparo Heurístico	6
4.2 Seleção, Cruzamento e Mutação	7
4.3 Elitismo e Laço Evolutivo	7
4.4 Cenário de Aplicação	7
5 Resultados	7
5.1 Topologia da Solução	8
5.2 Comportamento de Convergência	9
6 Conclusão	10
Referências	12

1 Introdução

Problemas de otimização combinatória ocupam posição central na ciência da computação aplicada, sobretudo quando se busca alocar recursos limitados de forma eficiente sobre estruturas de rede. Entre esses problemas, o da Cobertura Mínima de Vértices (MVC) destaca-se por sua simplicidade de formulação e por sua aplicabilidade direta em cenários práticos, tais como o posicionamento de sensores, câmeras de segurança ou pontos de controle em redes viárias, de telecomunicações ou de infraestrutura urbana [3].

Formalmente, dado um grafo não direcionado $G = (V, E)$, o problema consiste em encontrar o menor subconjunto $S \subseteq V$ tal que, para toda aresta $(u, v) \in E$, pelo menos um dos vértices u ou v pertença a S . A versão de decisão do problema – verificar se existe cobertura de tamanho menor ou igual a k – está entre os 21 problemas originalmente classificados por Karp como NP-completos [3], o que torna inviável, na prática, a busca por soluções exatas em grafos de grande porte por meio de algoritmos de força bruta ou programação dinâmica.

Diante dessa limitação, métodos heurísticos e metaheurísticos tornam-se alternativas naturais. Em particular, os Algoritmos Genéticos (AGs), inspirados nos mecanismos de seleção natural e herança genética [1, 2], têm sido amplamente empregados na resolução de problemas combinatórios NP-difíceis, por permitirem a exploração eficiente do espaço de soluções por meio de operadores de seleção, cruzamento e mutação, sem a necessidade de explorar exaustivamente todas as combinações possíveis.

Este relatório documenta a implementação de um Algoritmo Genético aplicado ao MVC, com a particularidade de incorporar um operador heurístico de reparo, executado após a inicialização da população e após cada operação de cruzamento e mutação. Esse operador corrige indivíduos inviáveis – isto é, soluções que deixam alguma aresta descoberta – de forma determinística, priorizando a inclusão do vértice de maior grau entre os dois extremos da aresta violada. O cenário de teste utilizado simula uma malha urbana de monitoramento, modelada como um grafo de Barabási-Albert, em que os vértices representam interseções viárias e a cobertura mínima corresponde ao número mínimo de ativos de segurança necessários para que toda via da malha seja monitorada por ao menos um ponto adjacente.

O objetivo deste trabalho é, portanto: (i) descrever a arquitetura do algoritmo implementado, com ênfase no operador de reparo heurístico; (ii) executar o algoritmo sobre a instância de teste gerada; e (iii) interpretar criticamente os resultados de convergência obtidos, sem extrapolar conclusões além do que os dados experimentais permitem sustentar.

2 Fundamentação Teórica

2.1 O Problema da Cobertura Mínima de Vértices

O MVC é intimamente relacionado a outros problemas clássicos da teoria dos grafos, como o Conjunto Independente Máximo e o Conjunto Dominante, sendo seu complemento

equivalente ao problema do Clique Máximo no grafo complementar [3]. Sua relevância prática decorre de sua capacidade de modelar situações em que é necessário garantir cobertura total de um conjunto de relações binárias (arestas) utilizando o menor número possível de elementos (vértices), o que se aplica, por exemplo, a problemas de vigilância de rede, alocação de servidores de monitoramento e posicionamento de sensores em infraestrutura crítica.

2.2 Algoritmos Genéticos

Algoritmos Genéticos constituem uma classe de metaheurísticas populacionais baseadas em analogias com a evolução biológica [1]. Cada solução candidata é representada por um cromossomo – no caso do MVC, um vetor binário cuja i -ésima posição indica a presença (1) ou ausência (0) do vértice i na cobertura. A cada geração, uma população de cromossomos é avaliada por uma função de aptidão (*fitness*), e os indivíduos mais aptos têm maior probabilidade de gerar descendentes por meio dos operadores de seleção, cruzamento (*crossover*) e mutação [2]. A repetição desse ciclo ao longo de várias gerações tende a conduzir a população a soluções de qualidade crescente, embora sem garantia formal de otimalidade.

Um desafio recorrente na aplicação de AGs a problemas com restrições – como é o caso do MVC, em que toda aresta deve ser coberta – é o tratamento de indivíduos inviáveis gerados por cruzamento ou mutação. A estratégia adotada neste trabalho é a do *operador de reparo*, que transforma determinísticamente qualquer indivíduo inviável em um indivíduo viável, em contraposição a abordagens baseadas em funções de penalidade.

2.3 Redes Livres de Escala e o Modelo de Barabási-Albert

Para a geração da instância de teste, foi utilizado o modelo de Barabási-Albert, que produz grafos por meio do mecanismo de ligação preferencial: novos vértices tendem a se conectar a vértices já bem conectados, resultando em uma distribuição de graus de cauda pesada (*power-law*), característica observada empiricamente em diversas redes reais, incluindo redes de infraestrutura, de transporte e sociais [4]. Esse modelo é adequado para representar, de forma simplificada, uma malha viária urbana na qual algumas interseções (cruzamentos centrais) concentram um número muito maior de conexões do que a maioria das demais.

3 Metodologia

A avaliação experimental seguiu as seguintes etapas: (i) geração de uma instância sintética de grafo por meio do modelo de Barabási-Albert; (ii) execução do Algoritmo Genético híbrido sobre essa instância; (iii) registro do histórico de *fitness* (melhor indivíduo e média populacional) a cada geração; e (iv) visualização gráfica da topologia resultante e da curva de convergência.

3.1 Codificação e Função de Aptidão

Cada indivíduo é representado por um vetor binário de comprimento $|V|$, em que o valor 1 na posição i indica que o vértice i integra a cobertura. A função de aptidão corresponde diretamente à cardinalidade do conjunto cobertura, ou seja, à soma dos bits do cromossomo:

$$f(\mathbf{x}) = \sum_{i=1}^{|V|} x_i \quad (1)$$

sendo o objetivo do algoritmo a *minimização* de $f(\mathbf{x})$, sujeito à restrição de viabilidade de que, para toda aresta $(u, v) \in E$, $x_u = 1$ ou $x_v = 1$.

3.2 Parâmetros Experimentais

O Quadro 1 resume os parâmetros do Algoritmo Genético utilizados na execução documentada no código-fonte analisado.

Tabela 1: Parâmetros do Algoritmo Genético utilizados na execução

Parâmetro	Valor
Tamanho da população (pop_size)	50
Número de gerações (generations)	150
Taxa de mutação (mutation_rate)	0,05 (5%)
Tipo de seleção	Torneio ($k = 3$)
Tipo de cruzamento	Um ponto de corte
Elitismo	Sim (1 indivíduo por geração)
Operador de reparo heurístico	Sim (baseado no grau dos vértices)

3.3 Instância de Teste

O grafo de teste foi gerado pela chamada `nx.barabasi_albert_graph(50, 3)`, produzindo uma rede com $n = 50$ vértices, na qual cada novo vértice se conecta a $m = 3$ vértices preexistentes. Para esse modelo, o número total de arestas é dado analiticamente por $E = m \cdot (n - m)$, o que resulta em $E = 3 \times (50 - 3) = 141$ arestas – valor exatamente coincidente com o reportado na execução, confirmando a correta geração da instância. O Quadro 2 apresenta as principais características estruturais do grafo.

Tabela 2: Características da instância de teste (malha urbana simulada)

Característica	Valor
Modelo de geração	Barabási-Albert
Número de vértices (n)	50
Parâmetro de ligação (m)	3
Número de arestas ($ E $)	141
Grau médio ($2 E /n$)	5,64

4 Implementação

O algoritmo foi implementado em linguagem Python, utilizando as bibliotecas `networkx` para manipulação e geração de grafos [5], `numpy` para operações numéricas [6] e `matplotlib` para visualização gráfica [7]. A arquitetura do código está organizada em quatro funções principais, descritas a seguir.

4.1 Operador de Reparo Heurístico

A função `repair_individual` é o componente central da estratégia híbrida adotada. Ela percorre todas as arestas do grafo e, para cada aresta cujos dois extremos estejam ausentes da cobertura (ambos com valor 0), inclui na cobertura o vértice de maior grau entre os dois, sob a justificativa de que vértices de maior grau cobrem, simultaneamente, um número maior de arestas adjacentes – estratégia análoga ao algoritmo guloso clássico para aproximação do MVC.

```

1 def repair_individual(individual, edges_indices, degrees):
2     repaired = list(individual)
3     for u, v in edges_indices:
4         if repaired[u] == 0 and repaired[v] == 0:
5             if degrees[u] > degrees[v]:
6                 repaired[u] = 1
7             else:
8                 repaired[v] = 1
9     return repaired

```

Código 1: Operador de reparo heurístico baseado em grau

Esse operador é aplicado em três momentos do ciclo evolutivo: (i) imediatamente após a geração aleatória da população inicial; (ii) após o cruzamento de dois indivíduos; e (iii) após a aplicação da mutação. Como consequência, *todo* indivíduo presente na população, em qualquer geração, corresponde a uma cobertura de vértices factível – isto é, nunca há indivíduos com

arestas descobertas –, eliminando a necessidade de funções de penalidade para tratar inviabilidade.

4.2 Seleção, Cruzamento e Mutação

A seleção de pais é realizada por torneio de tamanho $k = 3$ (função `tournament_selection`): três indivíduos são amostrados aleatoriamente da população e o de menor *fitness* (menor cobertura) é escolhido como pai. O cruzamento utiliza um único ponto de corte sorteado uniformemente entre as posições 1 e $|V| - 1$, gerando dois descendentes pela troca dos segmentos finais dos cromossomos dos pais. A mutação aplica, de forma independente a cada gene, uma inversão de bit ($0 \leftrightarrow 1$) com probabilidade igual à taxa de mutação configurada.

4.3 Elitismo e Laço Evolutivo

A função `genetic_algorithm_mvc` orquestra o ciclo completo: a cada geração, calcula-se o *fitness* de toda a população, registram-se o melhor valor e a média da geração nos vetores `best_history` e `avg_history`, e o melhor indivíduo da geração corrente é inserido diretamente na nova população (elitismo de um indivíduo). É importante notar que esse mecanismo de elitismo preserva o melhor indivíduo *da geração atual*, e não necessariamente o melhor indivíduo encontrado em toda a execução; o algoritmo mantém, em paralelo, uma variável `global_best` que é atualizada sempre que a geração corrente produz um resultado superior ao melhor já registrado, sendo esse valor global – e não o último valor da população – o efetivamente retornado e reportado como resultado final. Os demais indivíduos da nova população são gerados por sucessivas aplicações de seleção, cruzamento, mutação e reparo, até que o tamanho da população seja restabelecido.

4.4 Cenário de Aplicação

No código analisado, o problema é enquadrado em um cenário de monitoramento urbano: os vértices do grafo representam interseções viárias e as arestas representam trechos de via que devem ser cobertos por algum ativo de segurança posicionado em uma das interseções adjacentes. A heurística de reparo, ao priorizar vértices de maior grau, equivale, nesse cenário, a priorizar interseções com maior número de vias concorrentes para o posicionamento dos ativos – uma escolha que tende a reduzir o número total de ativos necessários, embora não garanta a obtenção da cobertura mínima global.

5 Resultados

A execução do algoritmo sobre a instância de teste produziu a saída textual reproduzida no Quadro 3, confirmando os parâmetros estruturais do grafo gerado e o tamanho da cobertura final obtida.

Tabela 3: Resultado final reportado pela execução do algoritmo

Métrica	Valor
Vértices do grafo	50
Arestas do grafo	141
Melhor cobertura encontrada	28 vértices
Percentual de vértices selecionados	56,0%

5.1 Topologia da Solução

A Figura 1 apresenta a topologia do grafo de teste, com os vértices selecionados para compor a cobertura mínima encontrada destacados em vermelho, e os demais vértices (não selecionados) em azul-claro.

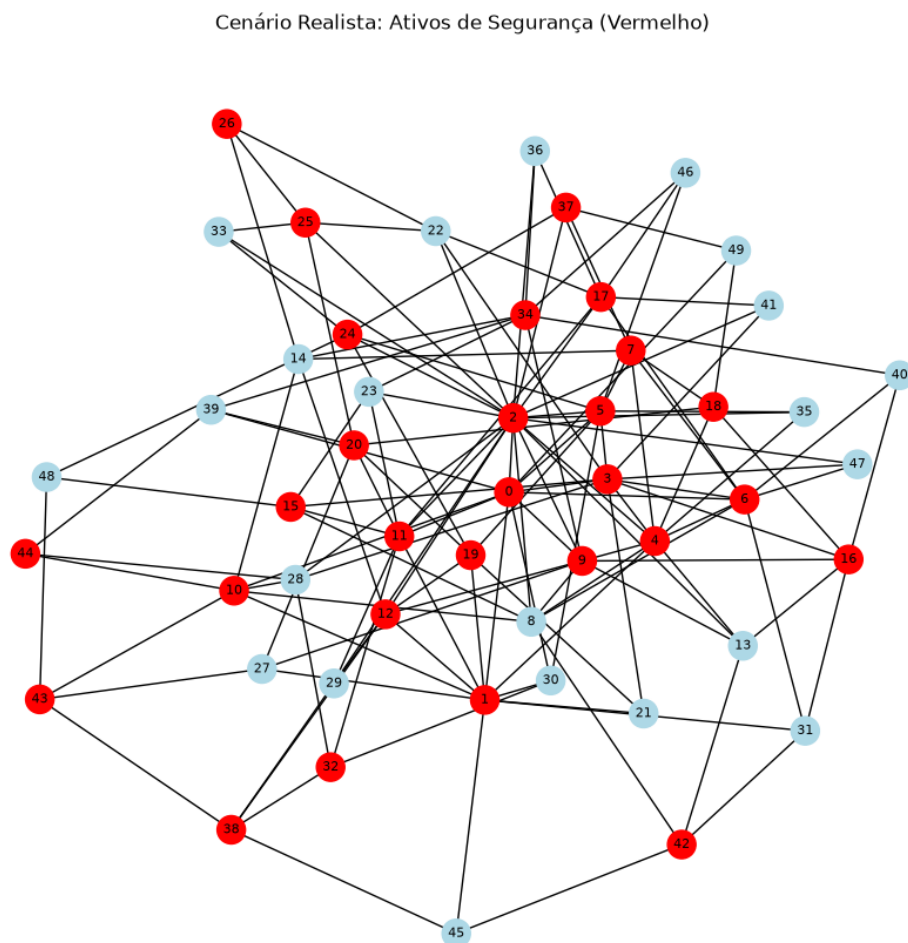


Figura 1: Topologia da malha urbana simulada com os ativos de segurança (cobertura) destacados em vermelho

A inspeção visual da Figura 1 permite observar que os vértices selecionados (em vermelho) concentram-se predominantemente entre os nós de maior grau de conexão, situados na região central do layout produzido pelo algoritmo de disposição em mola (*spring layout*). Esse padrão é consistente com o funcionamento do operador de reparo descrito na Seção 4.1, que prioriza explicitamente a inclusão de vértices de maior grau quando uma aresta encontra-se descoberta. Em contrapartida, vértices periféricos, de grau tipicamente baixo, permanecem fora da cobertura (em azul-claro), uma vez que suas arestas já se encontram cobertas pelo vértice central ao qual estão conectados, tornando sua inclusão redundante para a viabilidade da solução.

5.2 Comportamento de Convergência

A Figura 2 apresenta a evolução do *fitness* ao longo das 150 gerações, contrastando o melhor indivíduo de cada geração (linha azul contínua) com o *fitness* médio da população (linha laranja tracejada).

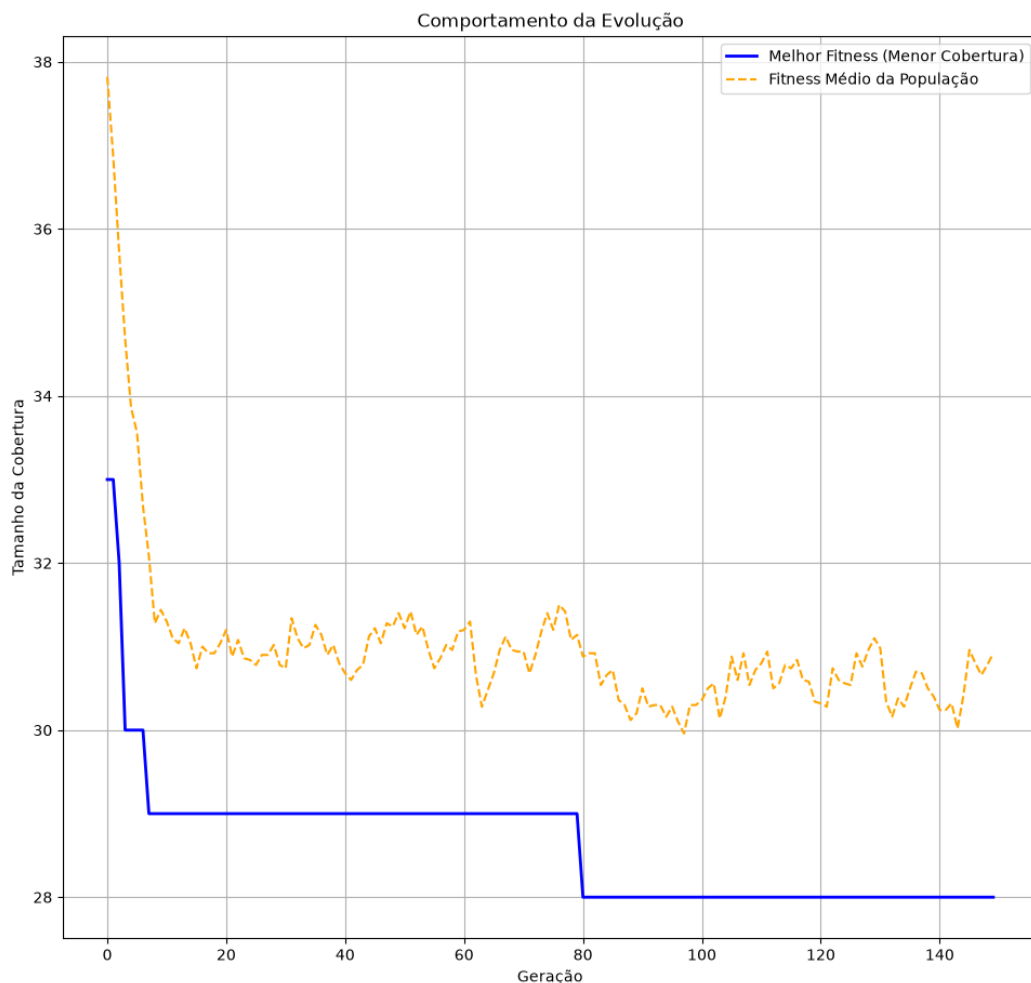


Figura 2: Curva de convergência do Algoritmo Genético: melhor cobertura e cobertura média por geração

A análise da Figura 2 revela um padrão de convergência caracteristicamente escalonado, típico de algoritmos genéticos com elitismo aplicados a problemas combinatórios discretos. Observam-se as seguintes fases:

1. **Fase inicial (gerações 0 a aproximadamente 8):** o melhor *fitness* parte de 33 – valor do melhor indivíduo já reparado da população inicial aleatória – e decai rapidamente, em degraus sucessivos, até estabilizar-se em 29. No mesmo intervalo, o *fitness* médio da população, que se inicia em torno de 37,8 (reflexo do custo adicional imposto pelo reparo sobre indivíduos iniciais com muitas arestas descobertas), cai abruptamente para a faixa de 30 a 31,5.
2. **Fase de estagnação (gerações aproximadamente 8 a 80):** o melhor *fitness* permanece constante em 29 por um intervalo prolongado de cerca de 70 gerações, enquanto o *fitness* médio oscila de forma estável entre 30 e 31,5, sem tendência de queda adicional. Esse comportamento sugere que a população, embora mantendo diversidade (evidenciada pela distância persistente entre a média e o melhor valor), não conseguiu produzir, por meio de cruzamento e mutação, um indivíduo estritamente melhor que o elite vigente durante esse intervalo.
3. **Melhoria tardia (a partir da geração aproximadamente 80):** o melhor *fitness* sofre uma nova redução, de 29 para 28, mantendo-se estável nesse valor até o término da execução, na geração 150. Esse evento ilustra a importância do operador de mutação na manutenção da capacidade de exploração do algoritmo, mesmo após um longo período sem progresso aparente, permitindo o escape de um aparente ótimo local.

Um aspecto relevante observado na Figura 2 é que a curva de *fitness* médio nunca converge para o valor do melhor indivíduo, mantendo um afastamento de aproximadamente 2 a 3 unidades durante praticamente toda a execução. Esse comportamento indica que a pressão seletiva do torneio de tamanho 3, combinada à taxa de mutação de 5%, foi suficiente para preservar diversidade genética na população, evitando convergência prematura, ainda que ao custo de uma velocidade de melhoria do valor médio relativamente lenta.

Por fim, cabe destacar que, por se tratar de um problema NP-difícil, os dados disponíveis não permitem afirmar que o valor de 28 corresponda à cobertura mínima global da instância, uma vez que não foi executado, no código analisado, nenhum método exato ou limitante inferior (como relaxação por programação linear) para fins de comparação. O valor de 28 deve, portanto, ser interpretado como a melhor solução factível encontrada pelo algoritmo dentro do orçamento computacional de 150 gerações, e não como um valor comprovadamente ótimo.

6 Conclusão

Este trabalho apresentou a implementação e a análise experimental de um Algoritmo Genético híbrido, dotado de operador de reparo heurístico, aplicado ao Problema da Cobertura

Mínima de Vértices, em um cenário hipotético de monitoramento de malha urbana. A incorporação do operador de reparo determinístico, baseado no grau dos vértices, mostrou-se eficaz em garantir a viabilidade de todos os indivíduos da população ao longo de toda a execução, eliminando a necessidade de mecanismos de penalização e simplificando a função de aptidão para uma simples contagem de vértices selecionados.

A execução sobre a instância de teste – um grafo de Barabási-Albert com 50 vértices e 141 arestas – produziu uma cobertura final de 28 vértices após 150 gerações, com população de 50 indivíduos. A análise da curva de convergência evidenciou um padrão típico de algoritmos genéticos aplicados a problemas combinatórios: queda rápida do *fitness* nas gerações iniciais, seguida de um platô prolongado de estagnação e, posteriormente, uma melhoria adicional viabilizada pela manutenção de diversidade populacional por meio da mutação. A manutenção de um afastamento persistente entre o *fitness* médio e o melhor *fitness* ao longo da execução sugere equilíbrio adequado entre exploração e exploração (*exploration* e *exploitation*), embora o longo intervalo de estagnação observado (aproximadamente 70 gerações) indique também um risco real de convergência prematura caso o número de gerações disponível fosse menor.

Como limitações do estudo, destacam-se: (i) a ausência de comparação com um limitante inferior exato ou com algoritmos de aproximação clássicos para o MVC, o que impede a quantificação do *gap* de otimalidade da solução obtida; e (ii) a realização de uma única execução, sem repetições com diferentes sementes aleatórias, o que impede a avaliação da variabilidade estatística do método. Como direções para trabalhos futuros, sugere-se a execução de múltiplas réplicas independentes, a comparação com heurísticas gulosas determinísticas e com relaxações de programação linear, bem como – conforme já indicado no próprio código-fonte analisado – a validação do algoritmo sobre instâncias de referência do tipo DIMACS, permitindo uma avaliação mais rigorosa de desempenho e escalabilidade.

Referências

- [1] HOLLAND, J. H. *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [2] GOLDBERG, D. E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading: Addison-Wesley, 1989.
- [3] GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.
- [4] BARABÁSI, A.-L.; ALBERT, R. Emergence of scaling in random networks. *Science*, v. 286, n. 5439, p. 509–512, 1999.
- [5] HAGBERG, A. A.; SCHULT, D. A.; SWART, P. J. Exploring network structure, dynamics, and function using NetworkX. In: *Proceedings of the 7th Python in Science Conference (SciPy2008)*, Pasadena, CA, 2008, p. 11–15.
- [6] HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020.
- [7] HUNTER, J. D. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007.